

A comparison of Peer Instruction and Process Oriented Guided Inquiry Learning-like pedagogies in teaching Software Testing and DevOps

Bhuvana Gopal
School of Computing
University of Nebraska-Lincoln
Lincoln, USA
bhuvana.gopal@unl.edu

Stephen Cooper
School of Computing
University of Nebraska-Lincoln
Lincoln, USA
stephen.cooper@unl.edu

Abstract—In this Research Full paper, we present a comparative study of using two pedagogical approaches, Peer Instruction (PI) and Process Oriented Guided Inquiry Learning-like (POGIL-like) in software testing. We discuss our results from multiple studies conducted over a period of four years, both in-person and online, using both PI and POGIL-like in undergraduate software engineering classrooms. Our particular focus is on the topics of unit testing, integration testing, and continuous integration. We compare learning outcomes and shifts in student attitudes compared to pure lecture-based instruction.

We included two studies conducted using PI in undergraduate software engineering classes. In these studies, we examined how PI can be used to teach the 3 topics mentioned above. They replicated their own studies over a period of two years and found consistently significant cognitive gains and positive affective shifts in both an in person and online class.

For POGIL-like, we included two studies in an undergraduate software engineering class - one in-person, and one online. The intervention design centered around the core tenets of POGIL: no prior information given to students (which is the exact opposite of a flipped classroom), and carefully crafted “models” and “activities”. POGIL-like utilizes “learning cycles”, and activity questions are crafted based on the Explore-Invent-Apply cycle combined with Directed/Convergent/Divergent question types, all aimed at encouraging students to leverage the collective knowledge of the group. These approaches, based on social constructivist concepts, have been well studied in STEM disciplines but are relatively recent arrivals in CS.

Our particular contribution is that we analyze both pedagogies in teaching software testing, using results from studies that utilized the same instructor and course materials. We analyze the similarities and differences between both pedagogical approaches, and rate their suitability in teaching software engineering and software testing.

Our primary research questions are:

- RQ1: How do students learn software testing and DevOps using PI and POGIL-like compared to pure lecture format classes?
- RQ2: What are the relative advantages and disadvantages for PI and POGIL-like from the instructor's perspective?

In each of the studies we analyzed, we collected and classified individual student responses from a pre- and post- course survey for each implementation of the course (lecture, PI or POGIL; in person or online; honors or regular). The pre- and post-surveys each included two components: a technical, content-related survey specifically designed to test student knowledge

and synthesis in unit testing, integration testing, and continuous integration, followed by a standard affective instrument aimed at gauging student attitudes and perceptions around the same topics. We adopted within-groups and between-groups Analysis of Variance techniques to determine statistical significance in our correlations.

In all 4 studies involving PI and POGIL-like, we found encouraging results showing that students had learned significantly better than pure lecture alone. In this paper, for both PI and POGIL-like, we discuss and compare overall correctness for each group, as well as cognitive and affective gains based on the pre- and post- surveys. We also classify the relative strengths and weaknesses of each pedagogy based on student feedback. We discuss the similarities in these approaches as well as implementation differences, in addition to impediments faced while implementing them, along with ideas to encourage increase more widespread adoption of these evidence-based pedagogies.

Index Terms—Software Engineering Education; Peer Instruction; Process oriented guided inquiry based learning; Active Learning; Cognitive gains; student affect; software testing; DevOps; pedagogies; comparison.

I. INTRODUCTION

Software testing and DevOps are integral to ensuring software quality in the industry. Unit testing, integration testing and continuous integration are three fundamental topics that software engineers need to master before they can be productive in the industry workforce. These topics are often difficult to teach undergraduate students since they require a testing mindset, with an eye for finding issues and troubleshooting them. These skills have been shown to be difficult for students to develop [1], [2]. Several ways of teaching software testing have been proposed and adopted, from code based approaches to gaming. Active learning approaches such as Peer Instruction (PI) and Process Oriented Guided Inquiry based Learning (POGIL) have been gaining popularity among computing educators over the past decade. While these approaches have shown to help students learn better in introductory and some mid level CS courses, their usage in undergraduate software engineering, particularly in software testing, have been limited. In this paper, we analyze our three recently published studies employing PI and POGIL-like (a version of POGIL)

in undergraduate software engineering classrooms, utilizing identical content and taught by the same instructor over a period of three years. We compare correctness gains from each approach for the same content, analyzed using similar statistical techniques, for results gathered from student responses to identical instruments, using identical pre and post test setups. We present the similarities and differences in each approach. We also elaborate on the challenges faced by the instructor in adopting and implementing these two pedagogies in a new domain - software testing and DevOps. We analyze the relative strengths and weaknesses of both pedagogies and examine their suitability in teaching software testing and DevOps in the undergraduate software engineering classroom.

The organization of the rest of this paper is as follows. Section II presents related work, Section III presents our research questions for this paper, Section IV presents a short overview of the implementations of PI and POGIL-like that was adopted in the classroom, Section V contains the analysis of our approaches from various aspects of student learning and instructor challenges, Section VI delineates some threats to the validity of this study, and Section VII concludes the paper.

II. RELATED WORK

A. Software testing and DevOps

Code reliability and its relationship to software testing knowledge was investigated in a study by Lemos et al. [3]. Studies by Edwards [4], and more recently by Bijlsma [5] observe that students are often ill prepared in testing their code, and emphasizes the need for real practical training to be incorporated in teaching software testing in undergraduate courses. Teaching software testing has lagged behind advancements in software testing in the industry, and this could be a contributing factor [6]. There are also inherent difficulties in teaching software testing [7] and learning software testing [1].

There are other studies explaining how to teach testing and DevOps and examining various approaches [8], but studies combining active learning approaches with teaching software testing are few and far between. Our study on student difficulties in studying software testing and DevOps [1] found that students struggled to identify entry and exit points into test cases; students had difficulty knowing what exactly to test, and how granular their tests should be. Deciphering if test cases satisfied product requirements was another student difficulty. Our other previous studies [?], [?], [9] have shown the efficacy of Peer Instruction (PI) and Process Oriented Guided Inquiry based Learning (POGIL)-like approaches in teaching software testing and DevOps.

B. Peer Instruction in CS and SE

For over a decade, Peer Instruction (PI) has proven to be an effective pedagogical approach for introductory CS education. In PI, students are encouraged to actively participate, discuss and analyze answers to multiple-choice questions [10]. Zingaro studied how PI performance in class expresses itself in a final exam [11].

PI has been used for the past decade in CS [10], [12]. Students and instructors alike have a favorable opinion of PI [10], [13], and help improve students' learning outcomes [14]. PI helps reduce failure rates [13], and gives instructors clues on which students are struggling, early on in the course [15]. PI has also been used in computer architecture, theory of computing and cybersecurity [16]. More recently, pilot and replication studies in software testing and DevOps found PI to be effective in helping students learn software testing and DevOps in both in person and online classes [17], [18].

C. Process Oriented Guided Inquiry based Learning (POGIL) in CS and SE

Process Oriented Guided Inquiry based Learning (POGIL) [19] is a group based collaborative learning approach. It relies heavily on a structured process, and involves teamwork. Student learning outcomes in non-CS STEM disciplines, and entry level CS courses have benefited from POGIL [19]. POGIL is a proprietary term [20], and we use the term "POGIL-like" to denote the pedagogical approach used for comparison with PI, in this paper.

POGIL-like uses predefined roles for each student in a team. Students are grouped teams of 4 members, and each student has a role to play in the team - Reflector, Manager, Presenter and Recorder [19]. Models are the content they need to learn, presented with figures, equations, code snippets, tables, animations, and plain text. Activities are based on each model, and are comprised of questions that probe student thinking, and practical exercises. POGIL, and by extension, POGIL-like, differs from other small group pedagogies primarily in the synchronous implementation of models, activities and role based groups, instead of having student groups complete worksheets [21].

Original studies in POGIL were conducted over two decades ago in chemistry and biomechanics [21], [22]. General chemistry courses where POGIL was used showed a reduction in DWF rates as well as better student performance [21]. These early studies researched academic confidence, academic performance, the role of instructor intervention, efficiency in large classrooms, and how collaborative learning using POGIL can be facilitated in classrooms [22]. Kussmaul [19] and Hu [23] have done pioneering work on POGIL in CS.

Social constructivism is the foundation upon which POGIL is built [19]. Models and activities are specifically designed to utilize POGIL uses a specific learning cycle, based on a sequence of exploring, inventing and applying [24]. POGIL studies in CS have been focused on introductory courses [19], [23], [25], [26]. Instructors at Furman University created inquiry-based assignments for CS2 [27]. POGIL's popularity in CS1 is evidenced by studies in this field [19], [23], [25], [28]. In software engineering, some process based POGIL activities like project management have been published [26]. Current work on POGIL in SE is focused on software development life cycles, project scheduling, story point estimation, version control and software release life cycles [20]. POGIL content has also been used for design patterns [29]. In cybersecurity

education, POGIL has started making inroads [30]. Not many studies exist on POGIL in software testing and DevOps. In comparing POGIL-like with PI, we included the pilot study which examined how the POGIL-like approach could be used to positively impact how students learn technical topics such as software testing and DevOps [9].

III. METHODS

In the studies we included, we initially focused on PI, and implemented it in the physical and virtual classroom [17], [18]. They then implemented POGIL-like in the physical classroom. All these courses were taught by a single instructor who received formal training on how to use PI and POGIL in the classroom, with the same content in the same order, utilizing the same instruments for assessing student learning outcomes, as well as the same pre-post questionnaires to gauge how students' cognition and affect was impacted. We base our comparison of PI and POGIL-like on these studies [9], [17], [18], [31].

The implementation of PI for all these studies was based on the in person PI work by Porter [10], [14]. The POGIL-like implementation followed standard POGIL guidelines presented by Kussmaul [19] and Hu [23]. At the beginning of the semester, we had students complete a survey in both PI and POGIL-like classrooms using a cognitive questionnaire [9]. There were 16 multiple choice questions, each with a single correct response, in the survey - 6 in unit testing, 3 in integration testing and 7 in continuous integration. Students answered the same survey questionnaire again after the intervention, towards the final week of the semester. At this point, students had been taught either through lectures or the POGIL-like pedagogy.

All our studies were reviewed and classified as exempt by our university's Institutional Review Board. The instructor of the course had attended multiple POGIL workshops (fundamental and activity writing) conducted through the POGIL project [20]. We created 4 POGIL-like activities for this study, 2 in unit testing, and 1 each in integration testing and continuous integration. In this paper, we present a comparison of the results from these studies in PI and POGIL-like, both from a cognitive as well as affective perspective, and examine the suitability of using each approach in teaching software testing and DevOps. The cognitive portion of the study utilized a survey containing the technical questions with multiple choice as well as open ended questions on unit testing, integration testing and continuous integration. The affective portion of the study utilized the validated instrument by Moskal and Hoegh [32]. We adopted within groups (pre- and post- PI or pre- and post- POGIL-like) and between-groups (post-PI vs post-lecture; post-POGIL-like vs post lecture) Analysis of Variance techniques to determine statistical significance in our correlations.

IV. RESEARCH QUESTIONS

Our particular contribution is that we have analyzed studies with both pedagogies in teaching software testing, using the

same instructor and course materials. We analyze the similarities and differences between both pedagogical approaches, and rate their suitability in teaching software engineering and software testing.

Our primary research questions are:

- RQ1: How do students learn software testing and DevOps using PI and POGIL-like compared to pure lecture format classes?
- RQ2: What are the relative advantages and disadvantages for PI and POGIL-like from the instructor's perspective?

V. ANALYSIS - RELATIVE STRENGTHS AND WEAKNESSES OF PI AND POGIL-LIKE

PI and POGIL-like are both small group collaborative active learning pedagogies. In looking at the studies across several semesters of teaching and learning unit testing, integration testing and continuous integration with PI and POGIL-like, we found that the instructor observed certain unique advantages and disadvantages to each pedagogy. We classify our analysis findings into the following subsections.

Table I provides a summary of the salient features of PI and POGIL-like based on our previous studies [9], [17], [18], [31]

TABLE I
COMPARISON OF PI AND POGIL-LIKE FOR TEACHING SOFTWARE TESTING AND DEVOPS

Topic	PI	POGIL-like
Classroom implementation	required reading quiz; two votes with student discussion between votes; instructor led discussion; total cycle time 5-6 minutes	E-I-A cycle; D/C/V questions; no votes; student discussion throughout session; adhering to time allotted for each section of the activity; total session time 40-50 minutes
Student roles	teams of 3-5 students; no distinct roles;	teams of 4-6 students; 4 distinct roles - manager, recorder, presenter, reflector
Instructor roles	active facilitation during the short peer discussions	active facilitation during the co-construction phase and discussion
Content Development	distractor based questions;	D/C/V questions, models, activities following E-I-A cycles
Student answer formats	multiple choice through clickers	any
Student prior preparation	flipped classroom - videos, readings before class	no prior preparation
Student participation	short, focused discussions	longer, involved working through a myriad of activities
Student cognitive learning outcomes and content mastery compared to pure lecture	Yes	Yes
Student affect improvement	gender and usefulness	gender, usefulness, interest, confidence and professionalism
Instructor challenges	balancing the flipped classroom and in class PI; formulating plausible distractors for PI questions; lack of time to teach all they want to teach in the topic	content creation for models and activities with E-I-A and D/C/V very time consuming; significant training required for the instructor to understand POGIL-like; facilitation during class challenging; assessing student process skills outcomes challenging; student group formation and role assignment challenging; identifying struggling students early is difficult due to student roles

A. Classroom implementation - PI vs POGIL-like

The classroom implementation of PI typically involves a flipped classroom approach, with required readings, followed

by an in class session with a mini lecture and several PI cycles. We summarize the implementation of PI that we are analyzing [17].

- (a) **Before class:** Students were given readings to complete before class. These readings typically introduced students to the basic concepts in the topic that was to be taught that day. A graded reading quiz was given at the beginning of class to ensure that students were motivated to complete the preparatory assigned readings.
- (b) **During class - first vote:** PI was implemented in both the in-person and virtual classroom in subsequent academic years. Mini lectures in person or on Zoom were followed by several PI cycles, consisting of instructor posing a question, students voting via clickers, and, depending on what percentage of students answered correctly as per Mazur's guidelines [?], a second vote [31]. Each vote was administered through a student response system called the clicker [33].
- (c) **Discussion in small groups (3-5 students):** In both the in person and online classes, small groups of 3-5 students discussed and shared their rationale for the choices they made. This could also entail students trying to convince each other to change their responses before voting again. For the in person class, students grouped together in tables, and for the online class, students were allotted into pre-assigned breakout rooms.
- (d) **During class - second vote (Q2):** Instructor poses the same question again, and students answer via clickers again.
- (e) **Instructor led discussion:** After the second vote, the instructor clearly explains the correct answer, while asking the class to share their answers and reasoning with them. Simon et al. [15] suggest showing students the voting results at this point, which we [17], [31] did. Zingaro and Porter [11] conducted a third vote at this point, which they did not, since the aim was to understand the influence of peer discussion on student learning gains.

Our POGIL-like pedagogy utilized the POGIL learning cycle [19]: 2 sessions in unit testing, 1 in integration testing, and 1 in continuous integration. There were no prior required readings, since POGIL-like emphasizes learning together during class.

Class session: Students worked in small groups of 4-6. There were no lectures or mini-lectures. Students studied the model and worked on a related activity or a set of activities [19].

At the end of class: The instructor solicited feedback on the topic and concepts learned from all students through a classwide group discussion. Students shared their reasoning and process of co-construction of concepts. The instructor clarified misconceptions and explained the correct answers in each of the activities.

B. Student and instructor roles - PI and POGIL-like

Student roles: In POGIL-like, our student teams were typically 4-6 students per team, assuming 4 distinct roles [34]:

Reflector, Manager, Presenter and Recorder. The role of the manager is to keep the team focused on the task at hand. Time management is the manager's responsibility. The Recorder writes down the solutions to the POGIL-like activities. The Presenter's role is to verbally present answers to the entire class at the end of the class. The Reflector's role is to examine how the session went, and to consider what could be improved and what could be retained in terms of process, similar to an Agile retrospective meeting. Each team member received a role card explaining their role and its responsibilities. Students were assigned new teams for each of the 4 sessions of our study, with roles being rotated each time. In contrast, student teams in PI were smaller (3-5 students per team), and did not have specific roles for individual team members.

Role of the instructor: In the POGIL-like classroom in [9], groups were carefully assigned by the instructor. The instructor was also able to observe closely how students interacted with each other using the prescribed roles [34]. Simultaneously, the instructor helped students develop and practice targeted process skills that were unique to POGIL-like but universal in software engineering: troubleshooting, communicating with other teams and within one's own team, collaboration and learning to listen to team members while expressing one's reasoning, to come together to solve problems [34]. The instructor was able to intervene in a timely fashion to provide additional guidance to students as needed [19], [35]. In the PI classrooms in [17], [18], the instructor was not actively involved during student discussion, but was rather a facilitator who was available to help. However, at the end of the second vote for each PI question, the instructor intervened and announced the correct answer and explained why the distractors were incorrect, before moving on to another mini-lecture or PI question cycle.

How do these implementations differ and where do they overlap? The primary similarity is in the small group aspect of both approaches. PI groups are 2-3 students, while POGIL-like uses the 4 defined roles and has a typical group size of 4 students. The lack of rigidity in roles within a PI group could lend itself to more unrestricted communication among group members, potentially resulting in livelier discussions. PI cycles are usually short; group discussions typically last between one to three minutes per PI question. Students can maintain focus and attention for such short periods of time better than during a lengthy lecture session.

Software testers and DevOps engineers typically need to communicate with various entities: the development team, the customer service team, the DevOps team, and product management. A well tested, continuously integrated piece of software depends on the smooth communication between all these teams in a timely manner, to be successfully released on time to the customer. POGIL-like groups follow a structured activity which is heavily process laden. The rigidity in group roles could, while helping maintain order and focus, limit all members from discussing their thoughts and ideas freely. However, this process based structure could help improve formal communication skills within students, which is specif-

ically helpful in the context of software testing and DevOps, which rely heavily on inter and intra team communication. However, students' focus and attention span needs to be taken into consideration while developing POGIL-like activities. A skilful combination of D/C/V questions within each E-I-A cycle can help maximize how students learn from the activity. We developed each E-I-A cycle to contain at least one D, C and V question. Some cycles had additional C and V questions, and our focus was to help students dig deeper into the topic while co-constructing knowledge and concepts.

C. Content development process for PI vs POGIL-like

When the instructor developed PI questions for each topic, the process that was adopted was based on the following steps:

- Identify broad student learning outcomes for the topic.
- Refine each outcome to correspond to a sub topic that can be taught in one class session.
- Identify how many PI cycles in each session, accommodating mini-lectures as needed.
- Identify a potential PI question for each PI cycle.
- Formulate distractors for each potential PI question.
- Design multiple choice questions including code tracing, code rearranging and Parson's problems.

This process involves several iterations of question formulation and refinement. It is important to note here that PI questions that we used were formulated by the instructor to help students achieve technical learning outcomes. There were no process / soft skill related aspects to the PI questions.

POGIL-like emphasizes the activity more than just the questions, unlike PI. Each POGIL-like activity takes a lot more time to craft than a set of PI questions. The steps involved in creating a POGIL-like activity are as follows:

- Identify broad student learning outcomes for the topic.
- Sub-classify the learning outcomes into Content Learning Outcomes (CLO) and Process Learning Outcomes (PLO).
- Group relevant PLOs and CLOs together to correspond to a sub topic that can be taught in one class session.
- Determine the time available in each class session for the POGIL-like activity. Remember, students come into the classroom without any prior readings or lectures, and will be constructing knowledge on the topic using the model and activities provided.
- For each combination of PLO/CLO, or for individual PLOs and CLOs intended to be addressed in the class session, create a POGIL-like model.
- Create one or more activities for each model. Each activity should contain a set of questions, which could be multiple choice/short answer, fill in the blanks, code tracing, match the right response, or Parson's problems.
- Rearrange/add/eliminate activities for each CLO and PLO as needed, to fit the time limit.
- Time each activity and mark the start and end times.
- Allocate a few minutes for instructor led classwide discussion at the end of each model/activity combination.

POGIL-like model development and activity development are considerably more complex than PI question development. We describe the process below.

D. Comparing POGIL-like Activity development to PI question development

POGIL-like activities consist of models and activities. Models are non text representations of content, typically presented using figures, tables, pictures and action verbs. The content that would be presented in PI during the flipped classroom [36], combined with the actual lectures, were the typical content presented using models. Three types of questions were used - Directed (D), Convergent (C) and Divergent (V) questions [9]. Directed questions are knowledge based and require no further thinking beyond the obvious answers contained in the model. Convergent questions are more difficult, requiring students to converge multiple direct concepts in their minds. Divergent questions require the maximum effort, and involve students applying concepts in situations and contexts that are vastly different from the original context in which they were learned. While convergent questions often lead to similar answers from various student teams, divergent questions often lead to vastly different answers from each team, in contexts that are much broader than the scope of convergent questions [20]. Each of these D/C/V questions were nudged students towards either Exploration (E), Concept Invention (I) or Application (A) [37]. We provided a few models for each POGIL-like activity [9]. Each model was followed by one or more activities. Each activity contained some combination of D/C/V questions, that spur the students to think in a sequence of E-I or E-A or I-A.

As an example, an I/D question in a POGIL-like activity would be a concept invention question, where students piece together a new concept, but is also directed, which means that the new concept comes from having the students recall some knowledge that was directly presented to them in the model pertaining to that activity. Each POGIL-like session should also have specific Content Learning Objectives (CLO) and Process Learning Objectives. We designed each activity with 1 or 2 PLOs and 2-3 CLOs for each testing unit [9]. The total time allotted for each sessions was 40 minutes, split up between time for reading and discussing the model, and time for collectively answering questions based on the model. There were no prior readings, since POGIL-like relies heavily on students being exposed to content afresh in the class in their teams, and having them construct understanding together.

Each scripted activity contained questions that help students navigate a POGIL-like learning cycle [25], focusing on the development of not just content skills, but also process skills. Typically, each model-activity combination introduced a concept (for example, pre- and post conditions for unit testing), and facilitated students exploring new concepts and finally applying the newly explored concept in varied situations. Our student teams followed specific roles, combined with specific steps, and produced reports helped them develop process skills

through meta-cognition, in addition to emphasizing individual responsibility [25].

D/C/V questions are related to but distinct from the phases in E-I-A cycles, and were instrumental in facilitating student discussions during the class session. Exploration questions could be directly related to a piece of information and hence a directed question. But exploration could lead to convergence or divergence as well. Invention questions usually involve combining multiple concepts, leading to convergence. Application questions involve contexts outside the original context of instruction, and could be either convergent or divergent. They could also be open ended questions [19]. Questions on unit testing and integration testing specifically lend themselves to open-ended questions, utilizing discussions of scenarios and test cases, compared to introductory CS courses, where the focus is usually on concept learning [38], and we crafted our D/C/V questions that follow the E-I-A cycle described above to incorporate open ended questions that nudge students towards deeper thinking and understanding of the 3 topics we studied.

Our POGIL-like activities were developed based POGIL cycles. Each cycle had 3 distinct phases: Exploration (E), Invention (I) and Application (A) [19], [24]. This cycle is typically known as the E-I-A cycle. Each of our activities also utilized one or more questions from a sequence of Directed-Convergent-Divergent (D/C/V) questions [19]. The E-I-A cycle was intended to work as follows. The explore phase (E) involved location of trends or patterns in the data provided in the model. Students also generate hypotheses based on this exploration, and test them. Concept invention (I) is the phase where students use the generated knowledge from the explore phase to define new terms and concepts. Most importantly, this ensures that students do not start with a blank slate before concept introduction, but rather, have constructed some understanding [23]. The application phase (A), which is the final phase in the learning cycle, asks students to apply the new concept that was constructed in the previous two phases, in other contexts.

In PI, we formulated our questions on each topic to utilize the equivalent of directed and convergent questions, but not many divergent questions. This was due to the time constraint inherent to the PI approach- we completed each PI cycle within 5-6 minutes including both votes and peer discussion in between votes. This meant that each vote lasted anywhere between 45 seconds to 2.5 minutes, with approximately 2 minutes between votes for discussion. Too many questions might discourage students; too few might not help them make enough important connections [19]. Easy questions give students confidence, but might make them complacent; difficult questions might discourage them. POGIL-like questions should be built on open-ended questions that follow Blooms Taxonomy [39]. Since there is a combination of easy, difficult, direct, convergent and divergent questions, students may struggle to manage their time and effort to each question [19].

E. Student answer types and formats

Working through POGIL-like activities involves answering questions with distractors, similar to PI questions. However, unlike PI, where student responses are collected through clickers, in POGIL-like, the activity exercises can contain multiple forms of questions after each model. Clickers limit the kind of questions that can be asked, to what can be answered within a set of multiple choice answers, be it Parson's problems, code tracing or filling in the missing answer. In POGIL-like, activity exercises can be in any format. In addition to the formats that are suitable for PI, POGIL-like activities can include code writing, reflections on reasoning, short answers, long answers, filling in missing code pieces, exception handling, and describing DevOps pipeline steps. POGIL-like lends itself well in helping students learn the process heavy steps involved in writing integration tests and connecting them to a DevOps pipeline.

F. Student participation

In POGIL-like, one can argue students are more visible to the instructor, simply because of the much longer duration of each class session. In our study [9], the instructor was able to closely observe students throughout each session, walking around the classroom and facilitating discussions and checking to see if teams were on track to complete activities on time. Some students took their time to warm up to the activities, especially the ones requiring discussion. Some students did not participate actively at all, and took cover behind the students that did. With PI, it was observed that students participated more uniformly, especially since their discussions were timed, short and focused on a specific question. The instructor was still able to facilitate discussions but the level of intervention by the instructor was lower.

G. Student cognitive learning outcomes and content mastery

We utilized the published results of the individual pedagogies in unit testing, integration testing and continuous integration [9], [17], [18]. Table II presents a comparative look at the learning gains exhibited by students across PI and POGIL-like from the above studies, averaging values across multiple studies for a given pedagogical approach. We cited the study in the header row, for reference.

TABLE II
STUDENT PERCENTAGE RAW LEARNING GAINS ACROSS PI AND POGIL-LIKE

Topic	PI-online [18]	PI-in person [17]	POGIL-like in person [9]
Unit testing	58	55	37
Integration testing	62	57	50
Continuous integration	70	65	51
Overall	63	59	46

We see significant correctness gains with both approaches. Continuous integration seems to have the largest gains, followed by integration testing and unit testing. Prior exposure to unit testing through other courses might be a factor in these

differences. While the correctness gains are comparable between PI in person and PI online, they are lower in POGIL-like (unit testing - 58% and 55% with PI vs 37%; integration testing - 62% and 57% with PI vs 50%; continuous integration - 63% and 59% with PI vs 46%). Based on the studies that reported these values, there were statistically significant differences in each of these groups versus pure lecture, so we know that both pedagogies work. But it is worth discussing why there are double digit percentages of students learning better from PI compared to POGIL-like. We think that, in comparing both pedagogies, time involved in answering questions could be a contributing factor. PI questions are typically answered within two votes, and the entire PI cycle for a question could be completed within 5-6 minutes. POGIL-like, on the other hand, uses entire class sessions, and is a more sustained exercise in learning the topic at hand. There are several process oriented elements that POGIL-like involves in its approach. Students start with no prior preparation, unlike the PI approach that utilizes a flipped classroom approach. Students have to adhere to roles, which are absent in PI. Students have to work together to co-construct concepts from scratch. This process heavy approach helps students develop much needed process skills they have been found lacking [1], [2] - specifically problem solving, communication and critical thinking. It is likely that students have gained better process skills with POGIL-like, but the focus is not on content mastery as much as it is on co-construction of concepts from the ground up.

H. Student affect

We compared affective outcomes over 5 constructs, in the studies with PI and POGIL-like in software testing and DevOps: confidence, gender, usefulness, professionalism and interest [17]. Our POGIL-like intervention showed that students had statistically significant shifts in affective outcomes for all 5 constructs [9], while our equivalent in-person PI implementation showed an improvement in the gender and usefulness constructs [18]. This signifies that POGIL-like has the potential to increase students' confidence and interest in software testing, more than PI. The extended time-frame of the POGIL-like approach, which spanned the entire class session in our implementation, could have been instrumental in influencing their perceptions of their own confidence and interest in the field of software testing. Having worked through divergent questions in larger numbers than during PI, students in the POGIL-like class would have had more opportunities to apply the concepts they co-constructed, elsewhere besides the original context. This could have helped them see the professional value of said concepts. There is a definite benefit in employing POGIL-like in terms of student affect, according to our pilot studies. This is a significant initial result that needs further replication to establish its generalizability.

I. Instructor challenges

In our analysis of PI vs POGIL-like, the following ideas emerged. In PI, the primary challenge for instructors would be how to flip the classroom and balance content in a way

that provides maximum learning opportunities for students in class during PI. How much required reading is too much? Formulating plausible distractors for each PI question could also prove difficult and requires an investment of time. The most common challenge for instructors in PI is the mindset of adopting an active learning classroom, with the focus being shifted from "cover the content through lecture" to "help students discuss and participate in learning the content at a deeper level". Keeping track of student teams during PI discussions, and being available to help students during in class PI sessions in a timely fashion, can be a difficult task, especially given the short duration required for PI discussions. Trying to fit different types of exercises into content that can be voted on through a multiple choice question, can also require some finesse. On the other hand, PI as a pedagogy is easy to learn, and the time investment required in preparing PI materials is not very different from creating questions for quizzes and exams. Regarding instructors' perceptions of PI, work by Porter [40] sheds the most light, and we concur that instructors can find value in the PI pedagogy.

Our analysis of POGIL-like from an instructor's perspective based on our previous work [9] illuminated the following challenges, chief of which is preparation time, consistent with the findings by Hu [41]. POGIL-like requires a significant time commitment from the instructor, starting with the training they have to undergo. In our study [9] we found that the instructor underwent several POGIL training workshops, some lasting several days. Understanding the learning science based roots of POGIL-like and dedicating effort to create content for models and activities with E-I-A cycles and D/C/V questions can be very time consuming. Several hours of effort might be required to create a single POGIL-like activity, including the model. The instructor has to put themselves into the student's shoes for every student role. How does the reflector, who primarily serves as a retrospective analyzer, help co-construct concepts along with the manager, whose main goal is keep track of time? The instructor has to not only accommodate, but leverage the various roles in POGIL-like, to offer students an optimal learning experience during the POGIL-like session.

To have a successful POGIL-like session, the instructor has to not only shift their mindset from pure lecture based instruction to active learning, but has to also recognize and appreciate the constructivist foundation of POGIL-like as a pedagogy. Classroom facilitation for POGIL-like can be challenging. It might take students several sessions to get used to the format of the class. Roles may not be adhered to strictly during the first few sessions, since students may find it overwhelming to adapt to active learning, and follow specified roles. Some students may not be willing subscribers to the active learning paradigm, and might need some coaxing to get involved. We think the best way an instructor can motivate students to follow POGIL-like is to take the time to explain to them the evidence based benefits of the pedagogy, and encouraging them to ask questions.

Assessing student process skills outcomes requires careful thought and planning on the part of the instructor. The CLOs

and PLOs for each module need to be well defined, along with clear ways to assess student mastery of those learning outcomes. Another area that instructors might find challenging in POGIL-like, is student group formation and role assignment. However, it might be beneficial to rotate students through roles for each POGIL-like session. Students participated in 4 POGIL-like sessions in software testing and DevOps. Each student was rotated through each of the 4 roles once by the end of the 4 sessions [9].

A distinct advantage that PI possesses over pure lecture is its ability to help instructors identify struggling students early [11]. POGIL-like could be used by instructors to do the same. However, this process is complicated by the addition of student roles in teams. This could be a distinct challenge for instructors who wish to utilize POGIL-like to help them identify weaker students who might need a little more help than others.

Another significant challenge in teaching software testing and DevOps with PI or POGIL-like is the small number of studies providing PI questions and POGIL-like activities. This is a nascent field and it would be beneficial for instructors and students alike to have more research in this field.

VI. THREATS TO VALIDITY

In comparing PI and POGIL-like, there are inherent aspects of implementation that are difficult to compare directly. The impact of a two minute discussion in PI, as opposed to an hour long POGIL-like session with discussion opportunities woven into the activities, is a topic for further study, and as such, remains a threat to the validity of our comparison. POGIL-like being process heavy, might be a better fit in teaching process related aspects of continuous integration. However, our study on POGIL-like [9] did not measure specific process related learning beyond the stated PLOs in each activity such as communication, collective problem solving and teamwork. A direct comparison of students' technical learning outcomes and affective shifts, while comparable and statistically significant, does not measure the impact of the role-specific aspects of POGIL-like when compared to PI. To address these research issues and more, further work is needed.

VII. CONCLUSION AND FUTURE WORK

Based on the above analysis and discussion, we assert the following: In response to **RQ1**- "How do students learn software testing and DevOps using PI and POGIL-like compared to pure lecture format classes?", we conclude that both PI and POGIL-like are effective ways to teach unit testing, integration testing and continuous integration at the undergraduate level in software engineering. In response to **RQ2**- "What are the relative advantages and disadvantages for PI and POGIL-like from the instructor's perspective?", we conclude the following: PI is advantageous to get students to learn testing concepts quickly through short, focused peer discussions, while POGIL-like not only helps students learn testing and DevOps concepts, but also helps them analyze and apply those concepts in contexts other than the original learning context. POGIL-like also helps students develop their process learning objectives better since

the intervention is targeted to help students improve process skills along with technical skills. PI and POGIL-like both require instructor training and effort outside the classroom to conduct the intervention in class, but POGIL-like requires a lot more instructor training and time investment than PI.

Based on our comparative analysis of PI and POGIL-like, we conclude that both be effective pedagogies to teach software testing and DevOps. While both pedagogies have shown a statistically significant improvement in student learning outcomes and affect, more replication studies are needed to verify the suitability of these approaches in specific aspects of software engineering education. PI, with its focused, short, small group discussions, is more suitable for introducing concepts and short problem solving questions. POGIL-like, with its process heavy approach, is suitable to not only improve students' technical knowledge, but also enhance their teamwork and collaboration skills. As for technical content knowledge, POGIL-like is more suitable to foster students' skills in co-constructing concepts and applying them in contexts outside of what they learned those concepts in. In the future, we intend to study the content of student discussions in both PI and POGIL-like, and believe that these discussions could hold a significant place in understanding the inner workings of these pedagogical approaches better.

REFERENCES

- [1] B. Gopal, S. Cooper, J. Olmanson, and R. Bockmon, "Student difficulties in unit testing, integration testing and continuous integration: An exploratory pilot qualitative study."
- [2] B. Gopal, S. Cooper, and R. Bockmon, "Industry partners' reflections on undergraduate software engineering students: An exploratory pilot qualitative study."
- [3] O. A. L. Lemos, F. C. Ferrari, F. F. Silveira, and A. Garcia, "Experience report: Can software testing education lead to more reliable code?" in *2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2015, pp. 359–369.
- [4] S. Edwards, "Using software testing to move students from trial-and-error to reflection-in-action," in *Proceedings of the 35th SIGCSE Technical Symposium on Computer Science Education*, p. 2630.
- [5] A. Bijlsma, N. N. Doorn, H. Passier, H. Pootjes, and S. Stuurman, "How do students test software units?: Part one: Their natural attitude diagnosed." Open Universiteit Nederland, 2020.
- [6] W. E. Wong, A. Bertolino, V. Debroy, A. Mathur, J. Offutt, and M. Vouk, "Teaching software testing: Experiences, lessons learned and the path forward," in *2011 24th IEEE-CS Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 2011, pp. 530–534.
- [7] P. Clarke, J. Pava, D. Davis, F. Hernandez, and T. King, "Using wrest in se courses: An empirical study," in *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE 12)*, ACM, New York, NY, USA, p. 307312.
- [8] S. Krusche, L. Alperowitz, B. Bruegge, and M. O. Wagner, "Rugby: an agile process model based on continuous delivery," in *Proceedings of the 1st International Workshop on Rapid Continuous Software Engineering*, 2014, pp. 42–50.
- [9] B. Gopal and S. Cooper, "POGIL-like Learning in Undergraduate Software Testing and DevOps - A Pilot Study," in *Proceedings of the 27th ACM Conference on on Innovation and Technology in Computer Science Education Vol. 1*, ser. ITiCSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 484–490. [Online]. Available: <https://doi.org/10.1145/3502718.3524776>
- [10] L. Porter, C. Bailey Lee, B. Simon, Q. Cutts, and D. Zingaro, "Experience report: a multi-classroom report on the value of Peer Instruction," in *Proceedings of the 16th Annual Joint Conference on Innovation and Technology in Computer Science Education*, 2011, pp. 138–142.
- [11] D. Zingaro and L. Porter, "Tracking student learning from class to exam using isomorphic questions," in *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*, 2015, pp. 356–361.
- [12] B. Simon, P. Kinnunen, L. Porter, and D. Zazkis, "Experience report: CS1 for majors with media computation," in *Proceedings of the Fifteenth Annual Conference on Innovation and technology in Computer Science Education*, 2010, pp. 214–218.
- [13] L. Porter, C. Bailey Lee, and B. Simon, "Halving fail rates using Peer Instruction: a study of four computer science courses," in *Proceeding of the 44th ACM Technical Symposium on Computer Science Education*, 2013, pp. 177–182.
- [14] L. Porter, S. Garcia, J. Glick, A. Matusiewicz, and C. Taylor, "Peer Instruction in computer science at small liberal arts colleges," in *Proceedings of the 18th ACM Conference on Innovation and technology in Computer Science Education*, 2013, pp. 129–134.
- [15] B. Simon, S. Esper, L. Porter, and Q. Cutts, "Student experience in a student-centered Peer Instruction classroom," in *Proceedings of the Ninth Annual International ACM Conference on International Computing Education Research*, 2013, pp. 129–136.
- [16] N. Tan, G. Shoemaker, A. Gedi, J. Mache, and R. Weiss, "Applying a framework for creating and analyzing cybersecurity questions for Peer Instruction," vol. 33, no. 1. Consortium for Computing Sciences in Colleges, 2017, pp. 102–108.
- [17] B. Gopal and S. Cooper, "Peer Instruction in software testing and continuous integration," in *Proceedings of the 52nd SIGCSE Technical Symposium on Computer Science Education*, 2021, pp. 255–260.
- [18] —, "Peer Instruction in Online Software Testing and Continuous Integration: A Replication Study," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Software Engineering Education and Training*, ser. ICSE-SEET '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 199–204. [Online]. Available: <https://doi.org/10.1145/3510456.3514168>
- [19] C. Kussmaul, "Process oriented guided inquiry learning (pogil) for computer science," in *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education*, 2012, pp. 373–378.
- [20] NSF TUES 1044679 POGIL in Computer Science. (2021) CS-POGIL. Last Accessed July 2021. [Online]. Available: <http://cspogil.org/>
- [21] R. S. Moog and J. N. Spencer, *Process oriented guided inquiry learning*. American Chemical Society Washington, DC, 2008, vol. 994.
- [22] C. P. Bailey, V. Minderhout, and J. Loertscher, "Learning transferable skills in large lecture halls: Implementing a pogil approach in biochemistry," vol. 40, no. 1. Wiley Online Library, 2012, pp. 1–7.
- [23] H. H. Hu, M. Lang, C. Kussmaul, C. Mayfield, and T. Pirmann, "Guided inquiry learning in context: perspectives on pogil in cs," in *Proceedings of the 45th ACM Technical Symposium on Computer Science Education*, 2014, pp. 467–468.
- [24] M. R. Abraham, "Inquiry and the learning cycle approach," vol. 1. Pearson Prentice Hall: Upper Saddle River, NJ, 2005, pp. 41–52.
- [25] H. H. Hu and C. Kussmaul, "Promoting student-centered learning with pogil," in *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education*, 2012, pp. 579–580.
- [26] C. Kussmaul, C. Mayfield, and H. Hu, "Process oriented guided inquiry learning in computer science: The CS-POGIL & IntroCS-POGIL projects," in *ASEE Annual Conference and Exposition, Conference Proceedings*, 2017, pp. 1–7.
- [27] K. Abernethy, P. Gabbert, and K. Treu, "Inquiry-based computer science instruction: Some initial experiences," vol. 30, no. 3. ACM New York, NY, USA, 1998, pp. 14–17.
- [28] M. Jonas, "Lessons learned from integrating pogil into a CS1 course," vol. 34, no. 6. Consortium for Computing Sciences in Colleges, 2019, pp. 139–140.
- [29] P. C. Lotlikar and R. Wagh, "Using POGIL to teach and learn design patternsa constructionist based incremental, collaborative approach," in *2016 IEEE Eighth International Conference on Technology for Education (T4E)*. IEEE, 2016, pp. 46–49.
- [30] H. Alshaher, X. Yuan, and S. Khorsandroo, "Teaching Flooding Attack to the SDN Data Plane with POGIL," in *Proceedings of the 21st Annual Conference on Information Technology Education*, 2020, pp. 194–199.
- [31] B. Gopal and S. Cooper, "Peer Instruction in software engineering findings from fine grained clicker data," in *Proceedings of the 52nd SIGCSE Technical Symposium on Computer Science Education*, 2021, pp. 476–480.
- [32] A. Hoegh and B. Moskal, "Examining science and engineering students attitudes toward computer science," in *2009 39th IEEE Frontiers in Education Conference*, p. 16.
- [33] iClicker. (2021) Student Response Systems & Classroom Engagement Tools. Last Accessed July 2021. [Online]. Available: <https://www.iclicker.com/>
- [34] H. H. Hu and T. D. Shepherd, "Teaching CS1 with POGIL activities and roles," in *Proceedings of the 45th ACM Technical Symposium on Computer Science Education*, 2014, pp. 127–132.
- [35] D. M. Hanson, "Designing process-oriented guided-inquiry activities," 2005, pp. 1–6.
- [36] M. L. Maher, C. Latulipe, H. Lipford, and A. Rorrer, "Flipped classroom strategies for CS education," in *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*, 2015, pp. 218–223.
- [37] M. Hu, M. Winikoff, and S. Cranefield, "Teaching novice programming using goals and plans in a visual notation," in *Proceedings of the Fourteenth Australasian Computing Education Conference-Volume 123*, 2012, pp. 43–52.
- [38] V. Razmov and R. Anderson, "Pedagogical techniques supported by the use of student devices in teaching software engineering," in *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education*, 2006, pp. 344–348.
- [39] B. S. Bloom, *Taxonomy of educational objectives, Handbook 1: Cognitive domain*, 2nd ed. New York, NY, USA: Addison-Wesley Longman Ltd.
- [40] L. Porter, D. Bouvier, Q. Cutts, S. Grissom, C. Lee, R. McCartney, D. Zingaro, and B. Simon, "A multi-institutional study of Peer Instruction in introductory computing," in *Proceedings of the 47th ACM Technical Symposium on Computing Science Education*, 2016, pp. 358–363.
- [41] H. H. Hu, C. Kussmaul, B. Knaeble, C. Mayfield, and A. Yadav, "Results from a survey of faculty adoption of process oriented guided inquiry learning (pogil) in computer science," in *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*, 2016, pp. 186–191.